

Лабораторная работа №2

Прямые методы решения систем линейных алгебраических уравнений

Вариант №5b

Задание.

- 1) Решить заданную систему линейных алгебраических уравнений $Ax = b$ методом LU разложения с выбором главного элемента по столбцу.
- 2) Используя полученное ранее LU разложение вычислить обратную матрицу к исходной.
- 3) Вычислить определитель и найти число обусловленности исходной матрицы в кубической, октаэдрической и евклидовой нормах.

Замечание.

Для определения собственных значений матрицы (евклидова норма) необходимо воспользоваться итерационным методом вращений для матрицы $A^T A$.

Исходные данные.

В лабораторной работе №2 варианта №5b нам дана матрица, представленная ниже.

$$\begin{bmatrix} 8.4 & 6.1 & -1.9 & 0.4 \\ 0.1 & -0.3 & -2.5 & -8.7 \\ 4.9 & 9.5 & -7.9 & 1.8 \\ 0.2 & 1.9 & -9.9 & -8.0 \end{bmatrix}$$

В качестве вектора правых частей будем использовать значения обеспечивающие точное решение

$$\vec{x} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

Теоретическая справка.

1 LU разложение.

1.1 Факторизация.

Идея: решить систему $Ax = b$, где A — исходная матрица системы и x, b — векторы левой и правой части соответственно. Представляем матрицу A как произведение нескольких простых матриц: $A = A_1 * A_2 * \dots * A_p$, где $A_1, A_2, \dots, A_p$ — элементарные матрицы преобразований. Это упрощает решение системы.

Сначала решаем последовательно:

$$A_1 * (A_2 * \dots * A_p * x) = A_1 * y_1 = b \Rightarrow y_1 = A_1^{-1} * b$$

$$A_2 * (A_3 * \dots * A_p * x) = A_2 * y_2 = y_1 \Rightarrow y_2 = A_2^{-1} * y_1$$

...

$$A_{p-1} * y_{p-1} = y_{p-2} \Rightarrow y_{p-1} = A_{p-1}^{-1} * y_{p-2}$$

$$A_p * x = y_{p-1} \Rightarrow x = A_p^{-1} * y_{p-1}$$

$y_1, y_2, \dots$ — промежуточные векторы, которые получаются при решении.

1.2 $A = LU$.

$$L * U * x = b \Rightarrow U * x = y, L * y = b \Rightarrow y = L^{-1} * b, x = U^{-1} * y$$

$$L = \begin{bmatrix} l_{1,1} & 0 & 0 & 0 \\ l_{2,1} & l_{2,2} & 0 & 0 \\ l_{3,1} & l_{3,2} & l_{3,3} & 0 \\ l_{4,1} & l_{4,2} & l_{4,3} & l_{4,4} \end{bmatrix}, \quad U = \begin{bmatrix} 1 & u_{1,2} & u_{1,3} & u_{1,4} \\ 0 & 1 & u_{2,3} & u_{2,4} \\ 0 & 0 & 1 & u_{3,4} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Обозначе-

ния: $\det(1) = a_{11}, \det(2) = \det(|a_{11} \ a_{12}| |a_{21} \ a_{22}|), \dots, \det(n) = \det(A)$

Теорема. Пусть $\det(j) \neq 0$ ($j = 1, n$), тогда существует и единственно представление $A = LU$, где $l_{ii} \neq 0$ ($i = 1, n$).

2 Метод Гаусса (приведение матрицы к верхнему треугольному виду).

Прямой ход:

$$A^{(n)} = \begin{bmatrix} 1 & a_{1,2}^{(n)} & a_{1,3}^{(n)} & a_{1,4}^{(n)} \\ 0 & 1 & a_{2,3}^{(n)} & a_{2,4}^{(n)} \\ 0 & 0 & 1 & a_{3,4}^{(n)} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Метод Гаусса —

это метод решения систем

линейных уравнений путём последовательного исключения неизвестных —

Горт Александр и Юсупова Александра НМТ-1,2

приведения расширенной матрицы системы к верхнетреугольному виду (или ступенчатому), а затем обратной подстановки.

2.1 Прямой ход.

Преобразуем матрицу A и вектор b к верхнетреугольному виду с единичной диагональю.

2.2 Обратный ход

Начиная с последнего уравнения, находим значения переменных:

$$x_n = b_n^{(n)},$$

$$x_{n-1} = b_{n-1}^{(n)} - a_{n-1n}^{(n)} \cdot x_n,$$

...

$$x_1 = b_1^{(n)} - \sum a_{1j}^{(n)} \cdot x_j \quad (j=2..n)$$

3 Определение и свойства элементарных матриц перестановок.

Определение: элементарной матрицей перестановок P_{kl} называется матрица, полученная из единичной матрицы перестановкой k -ой и l -ой строк.

Пример:

$$P_{1,2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad P_{1,3} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad P_{2,3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \left. \begin{array}{l} \text{Сво} \\ \text{й-} \\ \text{ства:} \end{array} \right\}$$

1. $P_{kl} \times A$ — перестановка k -ой и l -ой строк матрицы A ;
2. $A \times P_{kl}$ — перестановка k -ого и l -ого столбцов матрицы A ;
3. $P_{kl} \times P_{kl} = E \Rightarrow P_{kl}^{-1} = P_{kl}$;
4. $P_{kl} \times P_{ij}$ — матрица перестановок.

Замечание 1:

$$\det(P * A) = (\pm 1) \det(L) * \det(U) = \pm \prod |l_{ii}| \quad (i=1..n)$$

где P — матрица перестановок, фиксирующая перестановки строк в методе Гаусса с выбором главного элемента; знак определяется числом перестановок.

Замечание 2:

$$A * X = E \Rightarrow X = A^{-1}$$

Также, если

$A * x_k = e_k = [0 \dots 0 \ 1 \ 0 \dots 0]^T$ (единица на k -м месте), $k = 1..n$,

то при $P * A = L * U$ имеем: $L * U * x_k = P * e_k$.

Решение (по пунктам задания).

Для решения поставленной задачи была разработана и реализована программа на языке высокого уровня Python.

В коде использованы следующие прямые численные методы линейной алгебры:

LU-разложение с частичным выбором главного элемента — основной прямой метод для решения систем линейных алгебраических уравнений (СЛАУ), обеспечивающий численную устойчивость за счёт перестановки строк;

Прямой и обратный ход подстановки — для последовательного решения треугольных систем $Ly = Pb$ и $Ux = y$;

Построение обратной матрицы через решение n СЛАУ с единичными векторами в правой части;

Вычисление матричных норм: кубической (L_1), октаэдрической (L_2) и евклидовой (L_3), последняя — на основе собственных значений матрицы $A^T A$, найденных методом вращений Якоби;

Оценка числа обусловленности $\kappa(A) = \|A\| * \|A^{-1}\|$ в трёх нормах для анализа чувствительности решения к погрешностям;

Проверка корректности вычислений через невязку $\|Ax - b\|$, точность разложения $\|PA - LU\|$ и близость произведения AA^{-1} к единичной матрице.

Все эти методы относятся к классу прямых, так как дают точное решение за конечное число арифметических операций (в отсутствие ошибок округления).

В процессе решения выводятся все промежуточные результаты с комментариями: текущие состояния матриц L и U , а также номера переставляемых строк при выборе главного элемента. Матрица U (верхняя треугольная с единичной диагональю) строится по методу Гаусса с частичным выбором главного элемента, а элементы матрицы L вычисляются по стандартным формулам LU-разложения. По окончании вычислений выводится ранг исходной матрицы.

Оценка правильности полученных результатов.

Для проверки корректности вычислений выполнена следующая верификация: Вычислена невязка $r = Ax - b$ — она должна быть близка к нулю (в пределах погрешности вычислений). Сравнение с точным решением: найдено абсолютное отклонение между полученным численным решением $x_{\text{расчёт}}$ и заданным точным решением

$$\vec{x} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

При правильно реализованном алгоритме разница не должна превышать машинной погрешности. Проверка соотношений: $PA = LU$ — умножение матриц L и U даёт исходную матрицу A , предварительно переставленную матрицей перестановок P .

$Ax = b$ — подстановка найденного решения в исходную систему.

$AA^{-1} = E$ — умножение исходной матрицы на её обратную должно давать единичную матрицу. Эти проверки подтверждают корректность реализации метода и отсутствие ошибок в вычислениях.

Краткие выводы.

Целью работы было применение LU-разложения с выбором главного элемента для решения СЛАУ. Метод позволяет эффективно решать системы $Ax = b$, особенно при многократном использовании одной матрицы A . Для заданной матрицы 4×4 было построено разложение $PA = LU$, найдена обратная матрица, вычислен ранг, определитель и числа обусловленности в нормах L_1 , L_2 и L_3 . Здесь L_1 — кубическая норма, L_2 — октаэдрическая норма, а L_3 — евклидова норма, равная квадратному корню из максимального собственного значения матрицы $A^T A$. Системы решались при известном точном решении

$$\vec{x} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

Точность подтверждена малой невязкой $\|Ax - b\|$, корректностью разложения $\|PA - LU\|$ и близостью AA^{-1} к единичной матрице. Результаты демонстрируют надёжность прямых методов для точного решения СЛАУ.

Программа успешно выполняет все требуемые операции: строит LU-разложение, вычисляет обратные матрицы, определители, ранги и числа обусловленности, а также проверяет точность решений. Результаты подтверждают, что код работает корректно.

Результаты работы программы.

=====

Обработка Матрица A1

Горт Александр и Юсупова Александра НМТ-1,2

=====

Шаг 2: перестановка строк 1 и 2

Шаг 3: перестановка строк 2 и 3

Определитель: 2143.875000

Ранг: 4

Исходная матрица A:

8.400000	6.100000	-1.900000	0.400000
0.100000	-0.300000	-2.500000	-8.700000
4.900000	9.500000	-7.900000	1.800000
0.200000	1.900000	-9.900000	-8.000000

Матрица перестановок P:

1.000000	0.000000	0.000000	0.000000
0.000000	0.000000	1.000000	0.000000
0.000000	0.000000	0.000000	1.000000
0.000000	1.000000	0.000000	0.000000

Матрица L:

8.400000	0.000000	0.000000	0.000000
4.900000	5.941667	0.000000	0.000000
0.200000	1.754762	-7.848968	0.000000
0.100000	-0.372619	-2.903306	-5.472671

Матрица U:

1.000000	0.726190	-0.226190	0.047619
0.000000	1.000000	-1.143058	0.263675
0.000000	0.000000	1.000000	1.079404

Горт Александр и Юсупова Александра НМТ-1,2

0.000000 0.000000 0.000000 1.000000

Обратная матрица A^{-1} :

0.216800 -0.145395 -0.169944 0.130719

-0.148093 0.273632 0.258365 -0.246847

-0.039311 0.197235 0.071542 -0.200362

0.018895 -0.182726 -0.031421 0.067590

Числа обусловленности:

Кубическая норма (L1): 17.737529

Октаэдрическая норма: 22.339170

Евклидова норма (L2): 11.749522

Вектор b ($A \cdot x$): 16.500000 -42.800000 7.400000 -57.700000

Решение x из LU: 1.000000 2.000000 3.000000 4.000000

Невязка $Ax - b$: 0.000000e+00

Проверка PA - LU: 1.776357e-15

Проверка $A \cdot A^{-1} - I$: 3.400058e-16

=====

Обработка Матрица A2

=====

Шаг 2: перестановка строк 1 и 3

Определитель: -5423.621000

Ранг: 4

Исходная матрица A:

Горт Александр и Юсупова Александра НМТ-1,2

-7.700000 -2.800000 0.400000 0.100000
4.600000 3.100000 7.900000 4.400000
-5.000000 1.100000 -9.600000 6.700000
4.400000 7.400000 -0.700000 -3.700000

Матрица перестановок P:

1.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 1.000000
0.000000 0.000000 1.000000 0.000000
0.000000 1.000000 0.000000 0.000000

Матрица L:

-7.700000 0.000000 0.000000 0.000000
4.400000 5.800000 0.000000 0.000000
-5.000000 2.918182 -9.622548 0.000000
4.600000 1.427273 8.254971 12.620615

Матрица U:

1.000000 0.363636 -0.051948 -0.012987
0.000000 1.000000 -0.081281 -0.628079
0.000000 0.000000 1.000000 -0.880007
0.000000 0.000000 0.000000 1.000000

Обратная матрица A^{-1} :

-0.158137 -0.015506 -0.015630 -0.051016
0.087204 0.055434 0.039108 0.139096
0.073170 0.069728 -0.044105 0.005032
-0.027489 0.079235 0.067974 -0.053699

Горт Александр и Юсупова Александра НМТ-1,2

Числа обусловленности:

Кубическая норма (L1): 7.508205

Октаэдрическая норма: 7.186867

Евклидова норма (L2): 3.755644

Вектор b ($A \cdot x$): -11.700000 52.100000 -4.800000 2.300000

Решение x из LU: 1.000000 2.000000 3.000000 4.000000

Невязка $Ax - b$: 7.105427e-15

Проверка $PA - LU$: 1.776357e-15

Проверка $A \cdot A^{-1} - I$: 2.220446e-16

Текст программы

```
# Лабораторная работа №2
# Прямые методы решения СЛАУ
# Вариант №5b и №17a
# Горт Александр и Юсупова Александра, НМТ-1,2

import math
from contextlib import redirect_stdout

# Глобальные параметры задачи

N = 4          # размерность матрицы
EPS = 1e-12    # порог для сравнения с нулём (машинная точность)

OUTPUT_FILE = ".venv/lab2_output.txt" # директория сохранения отчета

# Вывод матрицы
def matrix_print(A, name=""):
    if name:
        print(f"{name}:")
    for i in range(N):
        print(" ".join([f"{A[i][j]:.6f}" for j in range(N)]))
    print()

# Вывод вектора
def vector_print(v, name=""):
    if name:
        print(f"{name}: ", end="")
    print(" ".join([f"{v[i]:.6f}" for i in range(N)]))
```

```
print()

# Копирование матрицы
def matrix_copy(src, dst):
    for i in range(N):
        for j in range(N):
            dst[i][j] = src[i][j]

# Создание единичной матрицы
def identity_matrix(I):
    for i in range(N):
        for j in range(N):
            I[i][j] = 1.0 if i == j else 0.0

# Умножение матриц
def matrix_multiply(A, B, C):
    for i in range(N):
        for j in range(N):
            C[i][j] = 0.0
            for k in range(N):
                C[i][j] += A[i][k] * B[k][j]

# Транспонирование
def transpose(A, AT):
    """Транспонирование:  $AT = A^T$ ."""
    for i in range(N):
        for j in range(N):
            AT[j][i] = A[i][j]

# Кубическая норма (максимум из сумм модулей элементов по столбцам)
def norm1(A):
    mx = 0.0
    for j in range(N):
        s = 0.0
        for i in range(N):
            s += abs(A[i][j])
        if s > mx:
            mx = s
    return mx

# Октаэдрическая норма (максимум из сумм модулей элементов по строкам)
def norm_inf(A):
    mx = 0.0
    for i in range(N):
        s = 0.0
        for j in range(N):
            s += abs(A[i][j])
        if s > mx:
            mx = s
    return mx

# Метод вращений Якоби для поиска собственных значений (евклидова норма через  $A^T * A$ )
def jacobi_eigenvalues(S, lambda_max, lambda_min):
    max_iter = 1000
```

Горт Александр и Юсупова Александра НМТ-1,2

```
# Копирование исходной матрицы
A = [[S[i][j] for j in range(N)] for i in range(N)]

for _ in range(max_iter):
    # Ищем максимальный по модулю внедиагональный элемент A[p][q]
    max_off_diag = 0.0
    p, q = 0, 1
    for i in range(N):
        for j in range(i + 1, N):
            if abs(A[i][j]) > max_off_diag:
                max_off_diag = abs(A[i][j])
                p, q = i, j

    # Если все внедиагональные малы — матрица почти диагональна, выходим
    if max_off_diag < EPS:
        break

    # Угол поворота, обнуляющий элемент A[p][q]
    app, aqq, apq = A[p][p], A[q][q], A[p][q]
    theta = 0.5 * math.atan2(2 * apq, aqq - app)
    c, s = math.cos(theta), math.sin(theta)

    # Обновление строк/столбцов, отличных от p и q
    for i in range(N):
        if i != p and i != q:
            aip, aiq = A[i][p], A[i][q]
            A[i][p] = A[p][i] = c * aip - s * aiq
            A[i][q] = A[q][i] = s * aip + c * aiq

    # Обновление диагональных элементов и обнуление A[p][q]
    A[p][p] = c * c * app - 2 * s * c * apq + s * s * aqq
    A[q][q] = s * s * app + 2 * s * c * apq + c * c * aqq
    A[p][q] = A[q][p] = 0.0

    # После итераций собственные значения стоят на диагонали
    lambda_max[0] = A[0][0]
    lambda_min[0] = A[0][0]
    for i in range(N):
        if A[i][i] > lambda_max[0]:
            lambda_max[0] = A[i][i]
        if A[i][i] < lambda_min[0]:
            lambda_min[0] = A[i][i]

    # Защита от деления на ноль при вычислении числа обусловленности
    if lambda_min[0] < EPS:
        lambda_min[0] = EPS

# Решение прямым ходом: решение L * y = b
def solve_Ly(L, b, y):
    for i in range(N):
        s = 0.0
        for j in range(i):
            s += L[i][j] * y[j]
        y[i] = (b[i] - s) / L[i][i]

# Решение обратным ходом: решение U * x = y
def solve_Ux(U, y, x):
    for i in range(N - 1, -1, -1):
```

```
s = 0.0
for j in range(i + 1, N):
    s += U[i][j] * x[j]
x[i] = y[i] - s

# LU-разложение с выбором главного элемента по столбцу
def LU_decomposition(A_orig, L, U, P):
    # Копия исходной матрицы
    W = [[A_orig[i][j] for j in range(N)] for i in range(N)]

    # P — единичная матрица
    identity_matrix(P)

    # Обнуляем L и U; диагональ U сразу делаем единичной
    for i in range(N):
        for j in range(N):
            L[i][j] = 0.0
            U[i][j] = 0.0
        U[i][i] = 1.0

    # Счетчик перестановок строк для знака определителя
    permutations = 0

    for k in range(N):
        # Выбор главного элемента по столбцу k в текущей матрице W
        pivot = k
        max_abs = abs(W[k][k])
        for i in range(k + 1, N):
            if abs(W[i][k]) > max_abs:
                max_abs = abs(W[i][k])
                pivot = i

        # Если весь столбец ниже диагонали — матрица вырождена
        if max_abs < EPS:
            return False, permutations

        # Перестановка строк (если нужна)
        if pivot != k:
            # Меняем строки в рабочей матрице W и в матрице перестановок P
            W[k], W[pivot] = W[pivot], W[k]
            P[k], P[pivot] = P[pivot], P[k]

        # Перестановка элементов L (в столбцах j < k)
        for j in range(k):
            L[k][j], L[pivot][j] = L[pivot][j], L[k][j]

        permutations += 1
        print(f"Шаг {k + 1}: перестановка строк {k} и {pivot}")

    # Формирование k-го столбца матрицы L
    L[k][k] = W[k][k]

    # Формирование k-й строки матрицы U (j > k)
    for j in range(k + 1, N):
        U[k][j] = W[k][j] / L[k][k]

    # Исключение нижележащих строк в столбце k
    for i in range(k + 1, N):
        L[i][k] = W[i][k]
```

Горт Александр и Юсупова Александра НМТ-1,2

```
        for j in range(k + 1, N):
            W[i][j] -= L[i][k] * U[k][j]
        W[i][k] = 0.0 # явно обнуляем

# Проверка: диагональные элементы L не должны быть нулевыми
for i in range(N):
    if abs(L[i][i]) < EPS:
        return False, permutations

return True, permutations

# Основная обработка матрицы
def process_matrix(A_orig, name):
    print(f"\n{'=' * 50}")
    print(f"Обработка {name}")
    print(f"{'=' * 50}")

    # Создание матриц
    L = [[0.0] * N for _ in range(N)]
    U = [[0.0] * N for _ in range(N)]
    P = [[0.0] * N for _ in range(N)]

    # LU-разложение с выбором главного элемента по столбцу
    success, permutations = LU_decomposition(A_orig, L, U, P)
    if not success:
        print("Матрица вырождена!")
        return

    # Ранг матрицы: количество ненулевых строк в U
    rank = 0
    for i in range(N):
        zero_row = True
        for j in range(N):
            if abs(U[i][j]) > EPS:
                zero_row = False
                break
        if not zero_row:
            rank += 1

    # Определитель (s — число перестановок строк)
    sign = 1.0 if permutations % 2 == 0 else -1.0
    det = sign
    for i in range(N):
        det *= L[i][i]

    print(f"\nОпределитель: {det:.6f}")
    print(f"Ранг: {rank}")

    # Печать исходных и промежуточных матриц
    matrix_print(A_orig, "Исходная матрица A")
    matrix_print(P, "Матрица перестановок P")
    matrix_print(L, "Матрица L")
    matrix_print(U, "Матрица U")

    # Обратная матрица через n решений СЛАУ с единичными векторами
    A_inv = [[0.0] * N for _ in range(N)]
    I = [[0.0] * N for _ in range(N)]
    identity_matrix(I)
```

```
for col in range(N):
    # Единичный вектор
    e = [l[i][col] for i in range(N)]

    # Учитываем перестановку
    Pb = [0.0] * N
    for i in range(N):
        for k in range(N):
            Pb[i] += P[i][k] * e[k]

    # Решаем  $L*y = Pb$ , затем  $U*x = y$ 
    y = [0.0] * N
    x_col = [0.0] * N
    solve_Ly(L, Pb, y)
    solve_Ux(U, y, x_col)

    # Записываем результат в col-й столбец обратной матрицы
    for i in range(N):
        A_inv[i][col] = x_col[i]

matrix_print(A_inv, "Обратная матрица  $A^{-1}$ ")

# Числа обусловленности в трёх нормах
cond1 = norm1(A_orig) * norm1(A_inv) # кубическая
cond_inf = norm_inf(A_orig) * norm_inf(A_inv) # октаэдрическая

# Евклидова
AT = [[0.0] * N for _ in range(N)]
ATA = [[0.0] * N for _ in range(N)]
transpose(A_orig, AT)
matrix_multiply(AT, A_orig, ATA)

lambda_max = [0.0]
lambda_min = [0.0]
jacobi_eigenvalues(ATA, lambda_max, lambda_min)

cond2 = math.sqrt(lambda_max[0] / lambda_min[0])

print("Числа обусловленности:")
print(f"Кубическая норма (L1): {cond1:.6f}")
print(f"Октаэдрическая норма: {cond_inf:.6f}")
print(f"Евклидова норма (L2): {cond2:.6f}")

# Формируем вектор b
x_true = [1.0, 2.0, 3.0, 4.0]
b = [0.0] * N
for i in range(N):
    for j in range(N):
        b[i] += A_orig[i][j] * x_true[j]

vector_print(b, "Вектор b ( $A*x$ )")

# Решаем систему  $A*x = b$ 
Pb = [0.0] * N
for i in range(N):
    for k in range(N):
        Pb[i] += P[i][k] * b[k]

y = [0.0] * N
x_sol = [0.0] * N
```

Горт Александр и Юсупова Александра НМТ-1,2

```
solve_Ly(L, Pb, y)
solve_Ux(U, y, x_sol)

vector_print(x_sol, "Решение x из LU")

# Проверка невязки r
res = [0.0] * N
for i in range(N):
    s = 0.0
    for j in range(N):
        s += A_orig[i][j] * x_sol[j]
    res[i] = s - b[i]

res_norm = max(abs(r) for r in res)
print(f"Невязка Ax - b: {res_norm:.6e}")

# Проверка разложения
PA = [[0.0] * N for _ in range(N)]
LU = [[0.0] * N for _ in range(N)]
matrix_multiply(P, A_orig, PA)
matrix_multiply(L, U, LU)

diff1 = 0.0
for i in range(N):
    for j in range(N):
        diff1 = max(diff1, abs(PA[i][j] - LU[i][j]))
print(f"Проверка PA - LU: {diff1:.6e}")

# Проверка корректности обращения матрицы
AA_inv = [[0.0] * N for _ in range(N)]
I_prov = [[0.0] * N for _ in range(N)]
matrix_multiply(A_orig, A_inv, AA_inv)
identity_matrix(I_prov)

diff2 = 0.0
for i in range(N):
    for j in range(N):
        diff2 = max(diff2, abs(AA_inv[i][j] - I_prov[i][j]))
print(f"Проверка A*A^-1 - I: {diff2:.6e}")
print()

def main():
    # Исходная матрица A варианта №5b
    A1 = [
        [8.4, 6.1, -1.9, 0.4],
        [0.1, -0.3, -2.5, -8.7],
        [4.9, 9.5, -7.9, 1.8],
        [0.2, 1.9, -9.9, -8.0]
    ]
    # Исходная матрица A варианта №17a
    A2 = [
        [-7.7, -2.8, 0.4, 0.1],
        [4.6, 3.1, 7.9, 4.4],
        [-5.0, 1.1, -9.6, 6.7],
        [4.4, 7.4, -0.7, -3.7]
    ]

    # Вывод программы в файл отчёта
    with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
```

Горт Александр и Юсупова Александра НМТ-1,2

```
with redirect_stdout(f):  
    process_matrix(A1, "Матрица A1")  
    process_matrix(A2, "Матрица A2")  
  
print(f"Результаты сохранены в файл: {OUTPUT_FILE}")  
  
if __name__ == "__main__":  
    main()
```